



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

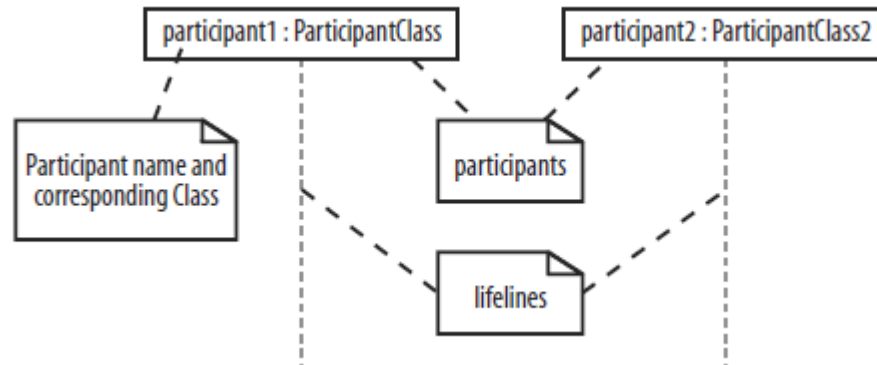
Sequence Diagrams

Francesco Guerra
francesco.guerra@unimore.it

Introduction

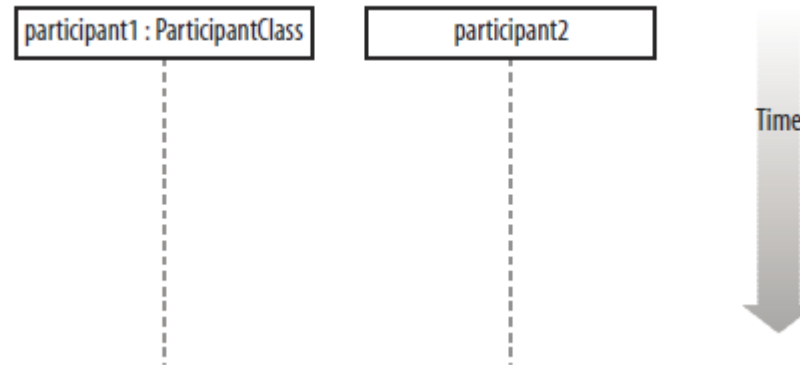
- ▶ Interaction diagrams describe how groups of objects collaborate in some behavior.
 - ▶ The most common UML form of interaction diagram is the sequence diagram.
- ▶ Typically, a sequence diagram captures the behavior of a single scenario.
 - ▶ The diagram shows a number of example objects and the messages that are passed between these objects within the use case.
 - ▶ With use cases and classes alone, you can't yet model *how* your system is actually going to its job.

Participants in a Sequence Diagram



- ▶ A sequence diagram consists of a set of participants.
 - ▶ A participant's lifeline shows that the element exists at that point in the sequence.
 - ▶ The lifeline is mainly relevant when a participant is created or deleted during the interaction.
- ▶ In UML 1.x: Participants were typically software objects in the traditional object-oriented sense.
 - ▶ Each object was an instance of a class, and its name was underlined to indicate this.
- ▶ In UML 2.0: The language is a general system modeling language.
 - ▶ Focus shifts to system parts interacting with each other rather than to software objects.

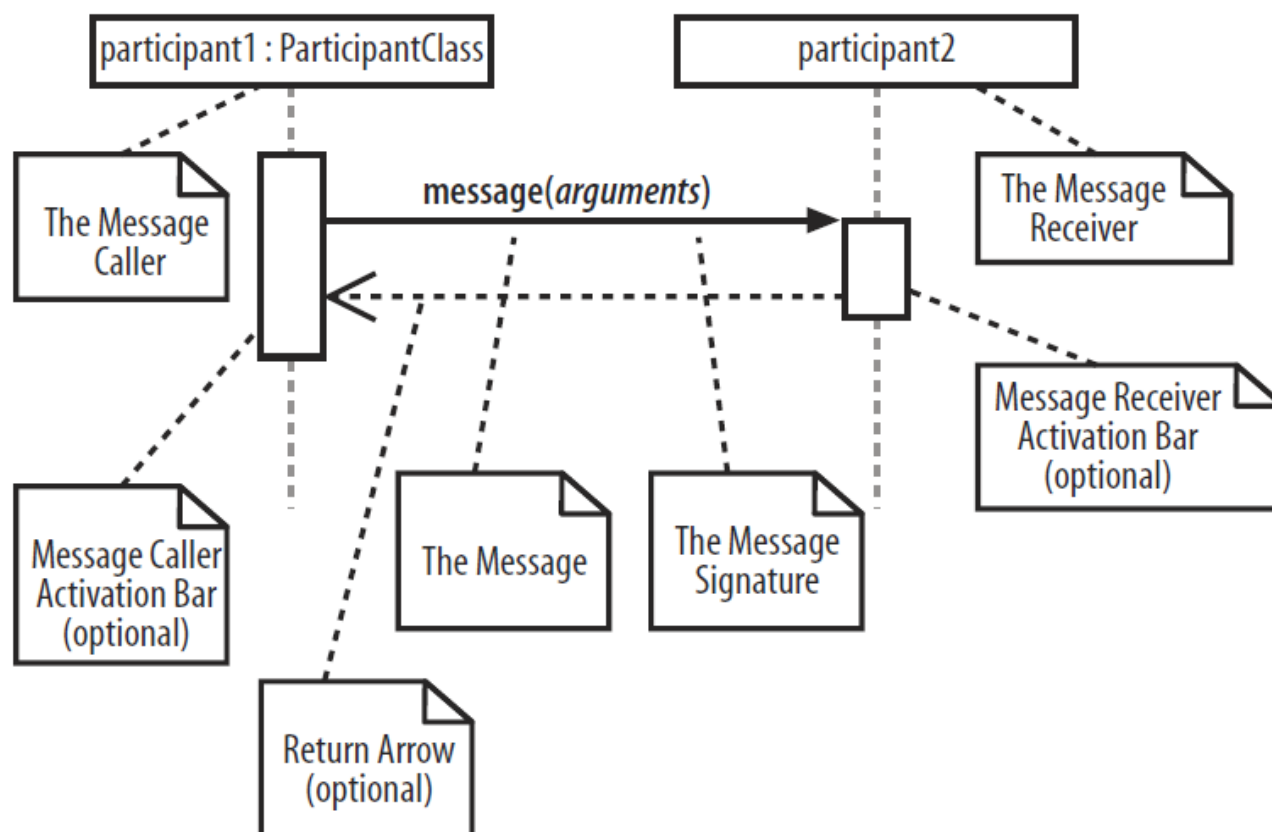
Time



Time on a sequence diagram is all about ordering, not duration.
How much of the vertical space the interaction takes up has nothing to do with the duration of time that the interaction will take.

Events and Messages

- ▶ The smallest part of an interactions is an event
 - ▶ Signals = messages

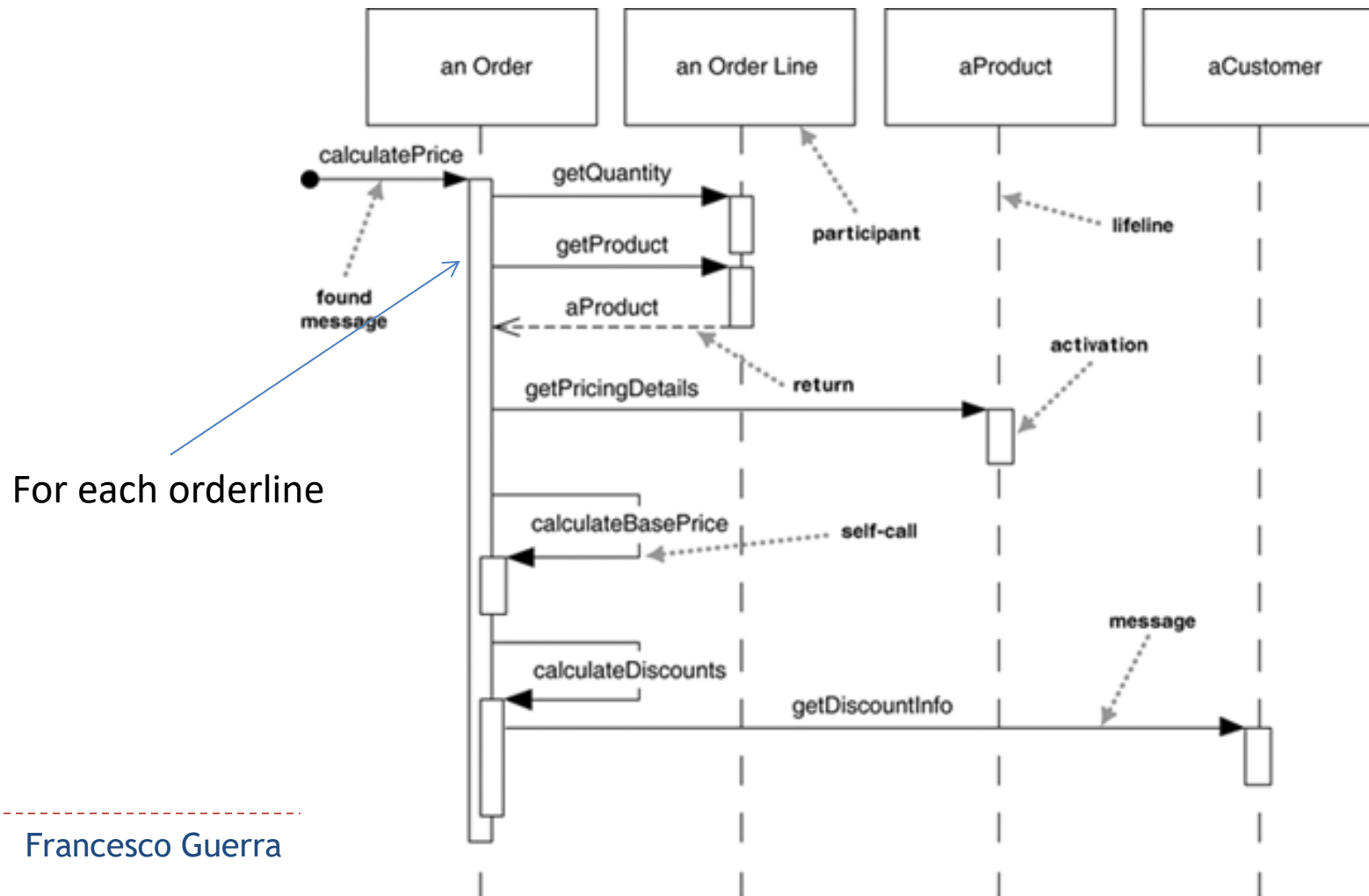


Messages

- ▶ Messages are specified using an arrow from the participant that wants to pass the message, the Message Caller, to the participant that is to receive the message, the Message Receiver
- ▶ Signature
 - ▶ *attribute = signal_or_message_name (arguments) : return_type*
 - ▶ The format of an argument is:
 - ▶ *<name>:<class>*
 - ▶ *myVar =doSomething(number1 : Number, number2 : Number) : ReturnClass*

Example

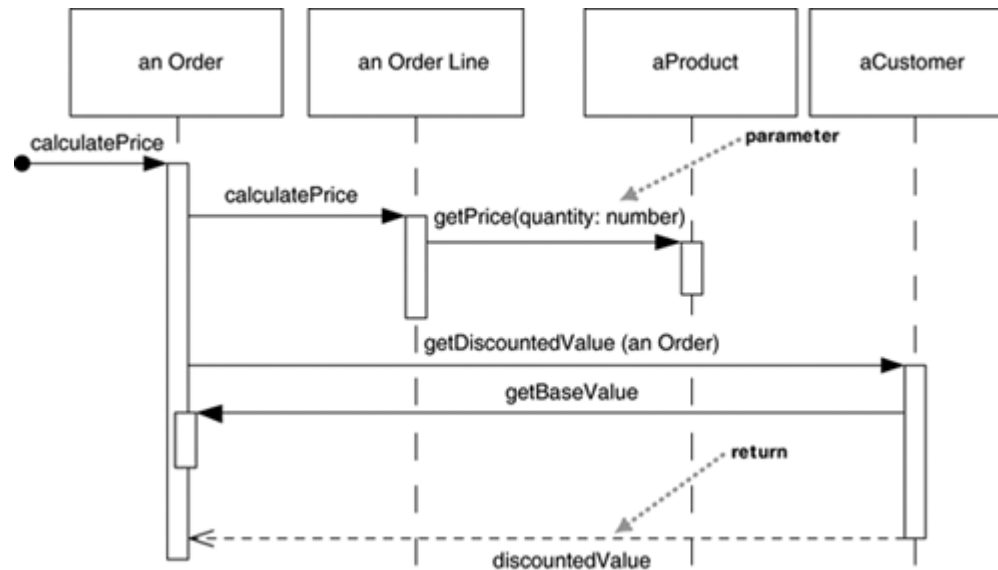
- ▶ We have an order and are going to invoke a command on it to calculate its price.



Comments on the example

- ▶ The diagram doesn't show everything clearly.
- ▶ The sequence of messages getQuantity, getProduct, getPricingDetails, and calculateBasePrice occurs for each order line, while calculateDiscounts is invoked once.
- ▶ Participants are named using the style anOrder.
 - ▶ Full syntax: name : Class
 - ▶ Both name and class are optional.
 - ▶ Keep the colon if the class is specified.
- ▶ Each lifeline has an activation bar showing when the participant is active. Activation bars are optional in UML.
- ▶ Return arrows are optional.
- ▶ The first message has no sending participant, coming from an undetermined source.
 - ▶ This is called a found message.

Example (2)

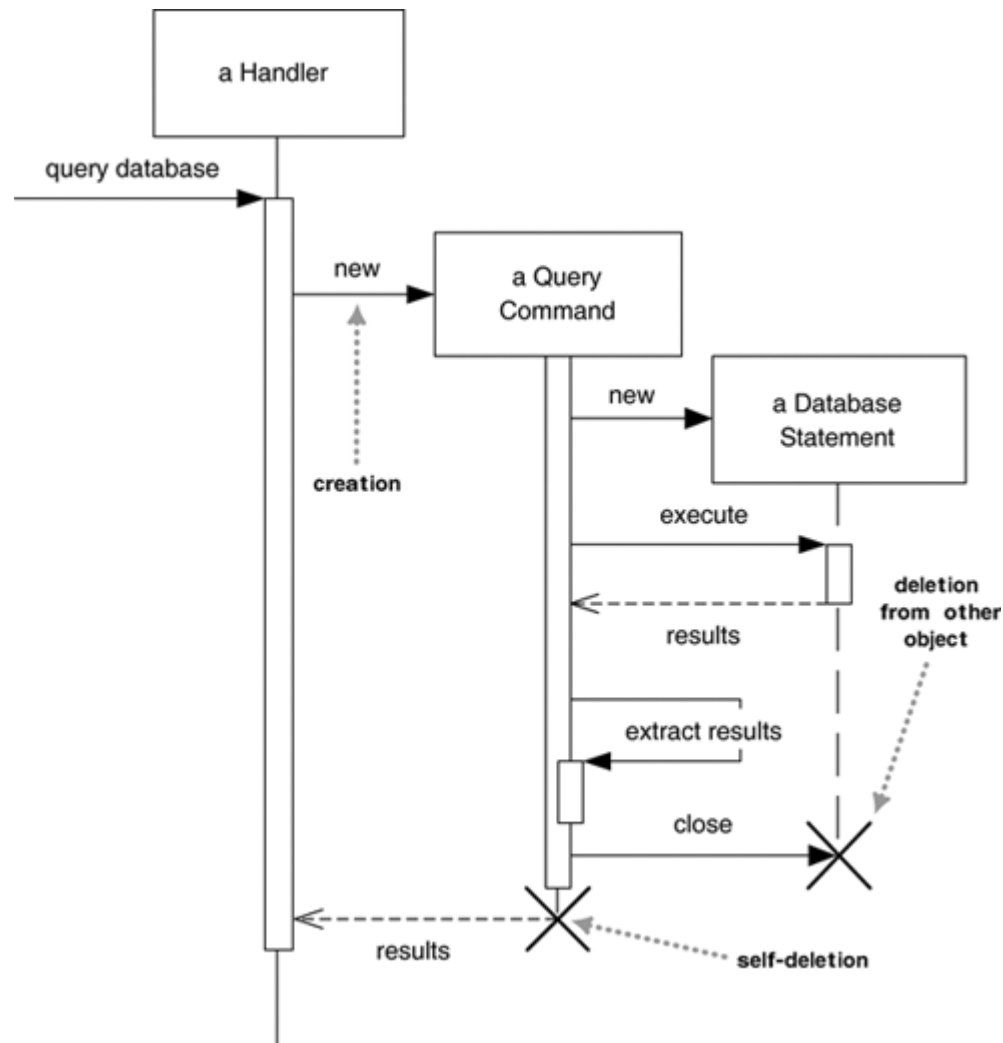


- ▶ These diagrams don't show details of algorithms but
 - ▶ they make the calls between participants crystal clear
 - ▶ give a really good picture about which participants are doing which processing.

Comments on the example (2)

- ▶ The diagram uses **distributed control**.
 - ▶ Processing is split among many **participants**, each performing a **fragment of the algorithm**.
- ▶ **Object-oriented principle**:
 - ▶ Data and the behavior that uses it should be **together in one place**.
- ▶ **Benefits of distributed control**:
 - ▶ Enables more use of **polymorphism** instead of **conditional logic**.
 - ▶ If **product pricing algorithms** differ for different types of products, **subclasses** can handle these variations.

Creating and Deleting Participants



Creating and Deleting Participants (2)

- ▶ **To create a participant:**
 - ▶ Draw the **message arrow** directly into the participant box.
 - ▶ A **message name** is optional (usually “**new**”).
 - ▶ If the participant immediately performs an action (e.g., a **query command**), start an **activation** right after the participant box.
- ▶ **Deleting a participant:**
 - ▶ Indicated by a **big X**.
 - ▶ A **message arrow into the X** shows one participant deleting another.
 - ▶ An **X at the end of a lifeline** indicates a participant deleting itself.
- ▶ **In a garbage-collected environment:**
 - ▶ Objects aren’t deleted directly.
 - ▶ Use the **X** to indicate when an object is **no longer needed** and ready for **collection**.
 - ▶ Also appropriate for **close operations**, signaling that the object is **no longer usable**.

Participant Names

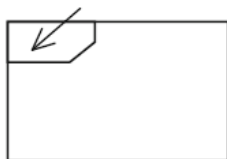
- ▶ Participants on a sequence diagram can be named in number of different ways, picking elements from the standard format:
- ▶ *name [selector] : class_name ref decomposition*

Example participant name	Description
admin	A part is named <code>admin</code> , but at this point in time the part has not been assigned a class.
: ContentManagementSystem	The class of the participant is <code>ContentManagementSystem</code> , but the part currently does not have its own name.
admin : Administrator	There is a part that has a name of <code>admin</code> and is of the class <code>Administrator</code> .
eventHandlers [2] : EventHandler	There is a part that is accessed within an array at element 2, and it is of the class <code>EventHandler</code> .
: ContentManagementSystem ref cmsInteraction	The participant is of the class <code>ContentManagementSystem</code> , and there is another interaction diagram called <code>cmsInteraction</code> that shows how the participant works internally (see "A Brief Overview of UML 2.0's Fragment Types," later in this chapter).

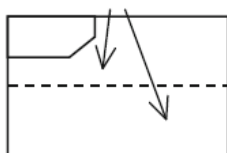
Combined Fragments

- ▶ *Combined fragments* (operators) are used to model various control structures explicitly.
- ▶ This enables you to describe a number of possible execution paths compactly and precisely.
- ▶ There are 12 different types of operators split into three groups:
 - ▶ Branches and loops
 - ▶ Concurrency and order
 - ▶ Filters and assertions

Operator

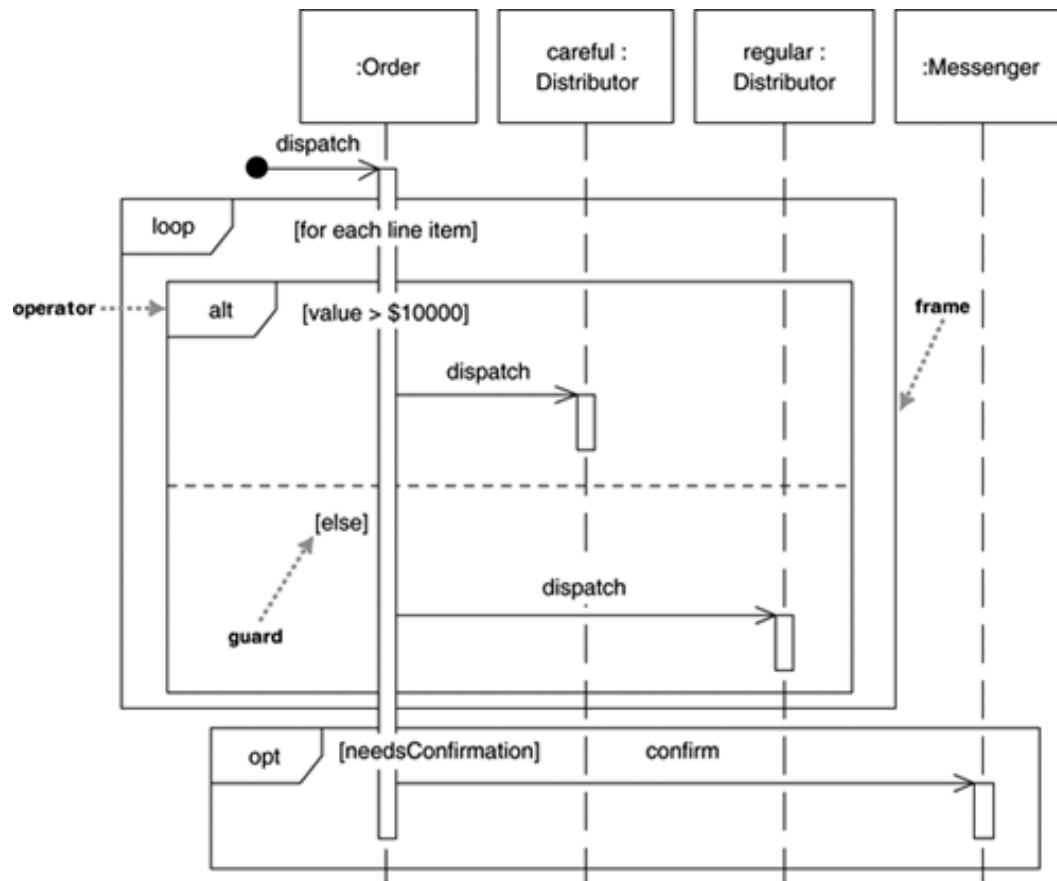


Operands



	Operator	Purpose
Branches and loops	alt	Alternative interaction
	opt	Optional interaction
	loop	Iterative interaction
	break	Exception interaction
Concurrency and order	seq	Weak order
	strict	Strict order
	par	Concurrent interaction
	critical	Atomic interaction
Filters and assertions	ignore	Irrelevant interaction parts
	consider	Relevant interaction parts
	assert	Asserted interaction
	neg	Invalid interaction

Loops, Conditionals, and other



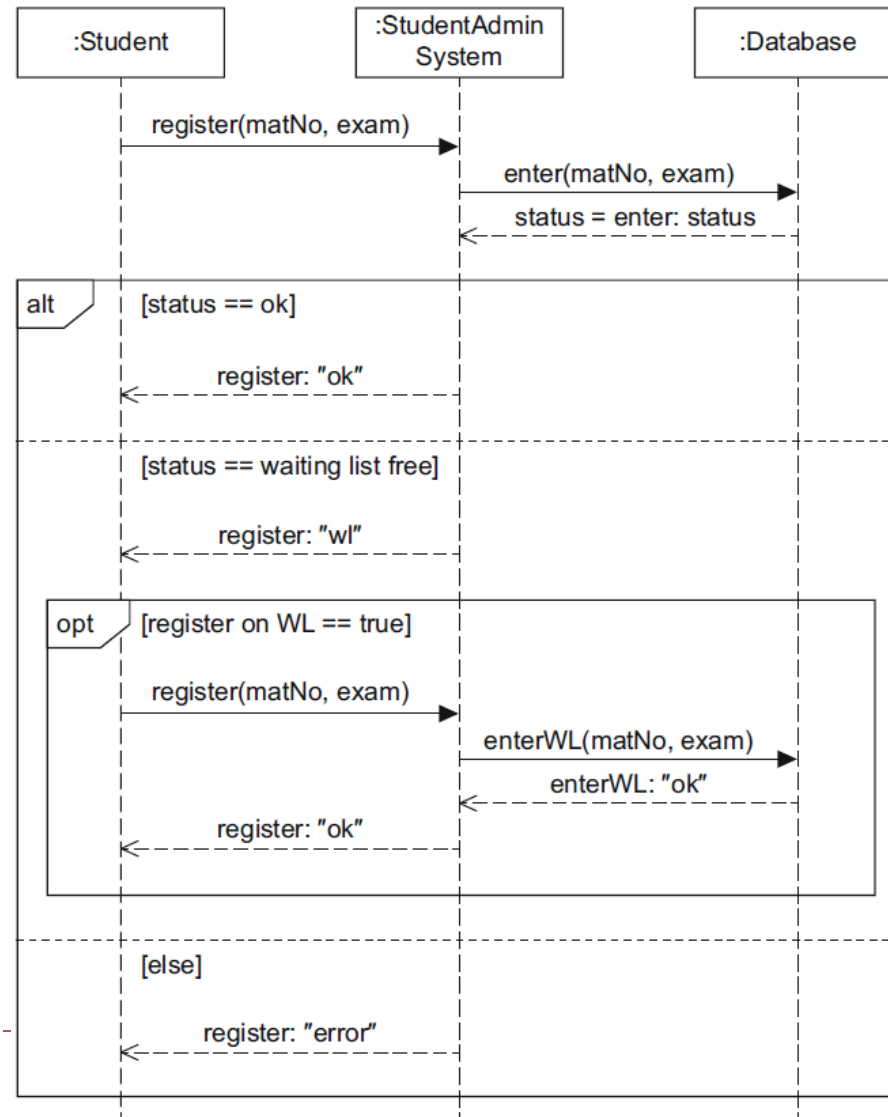
```

procedure dispatch
foreach (lineitem)
  if (product.value > $10K)
    careful.dispatch
  else
    regular.dispatch
  end if
end for
if (needsConfirmation)
  messenger.confirm
end procedure
  
```

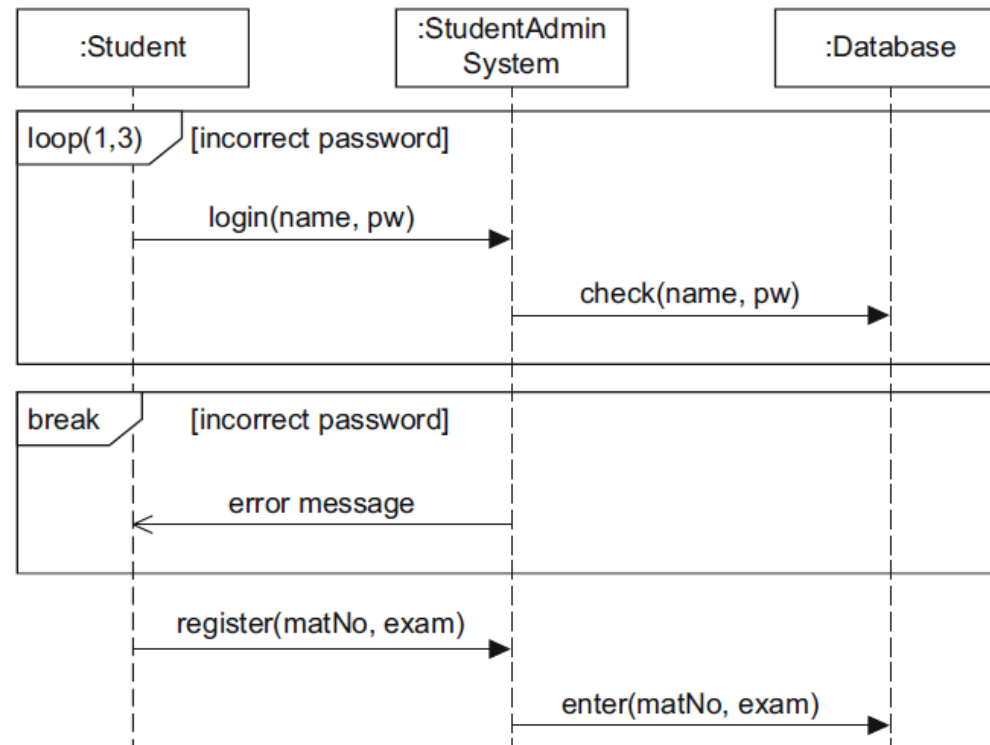
Branches and Loops

- ▶ Use an **alt fragment** to represent **alternative sequences**.
- ▶ An **alt operator** has at least **two operands**.
- ▶ Represents an **alternative path** in execution, similar to **switch statements** in programming languages (e.g., Java).
- ▶ **Guards** determine which path is executed:
 - ▶ Each operand has a **guard** (a **boolean expression** in square brackets).
 - ▶ If no guard is specified, **[true]** is assumed as the **default**.
 - ▶ If multiple guards are **true simultaneously**, this causes **indeterminism**.
 - ▶ Special guard **[else]** is **true** if no other condition is fulfilled.

Example



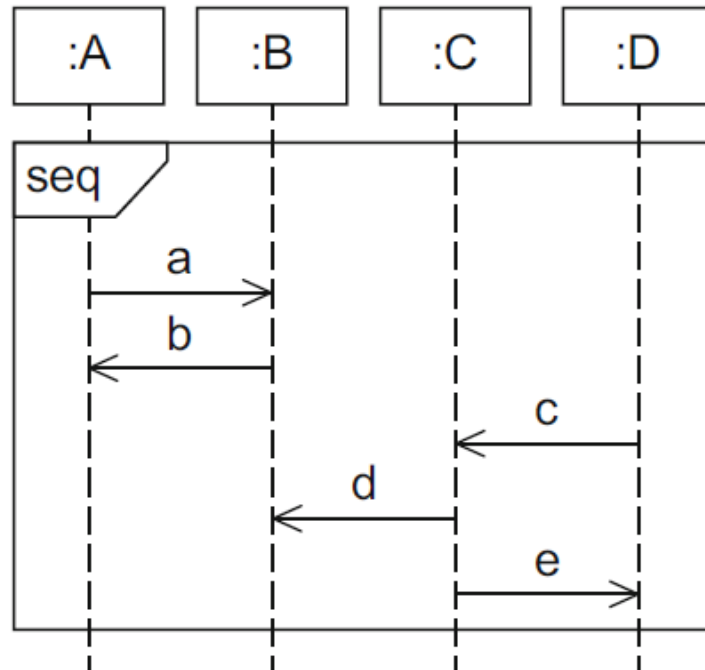
Example



Concurrency and order seq

- ▶ The **vertical axis** represents the **chronological order** of events, provided there is a **message exchange** between participants.
- ▶ **Combined fragments** allow explicit control of event order.
- ▶ **Seq fragment:**
 - ▶ Represents the **default order**.
 - ▶ Must have at least **one operand**.
 - ▶ Expresses **weak sequencing** according to UML:
 - ▶ Events **within each operand** are maintained in order.
 - ▶ Events **on different lifelines** from different operands may occur in **any order**.
 - ▶ Events **on the same lifeline** from different operands are **ordered**: events of the first operand occur **before** those of the second operand.
- ▶ **Using seq with break fragment:**
 - ▶ Messages from the seq fragment not yet executed are **skipped** if the **break condition** is true.
 - ▶ Execution continues in the **surrounding fragment**.

Example



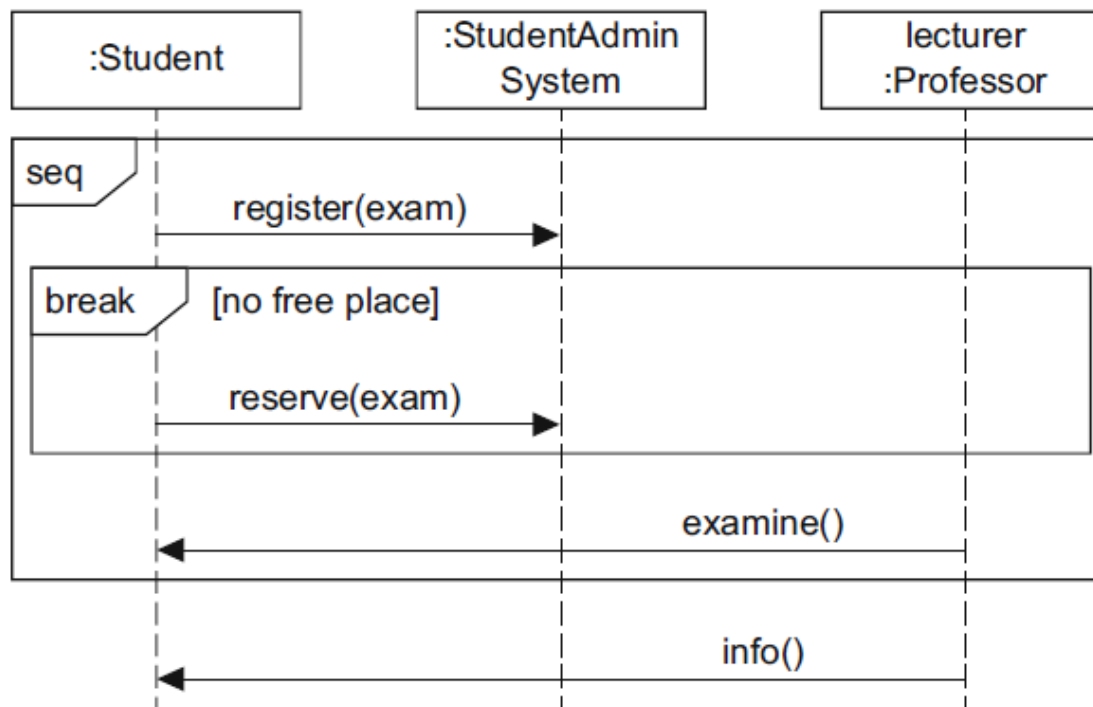
Traces:

T01: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$

T02: $a \rightarrow c \rightarrow b \rightarrow d \rightarrow e$

T03: $c \rightarrow a \rightarrow b \rightarrow d \rightarrow e$

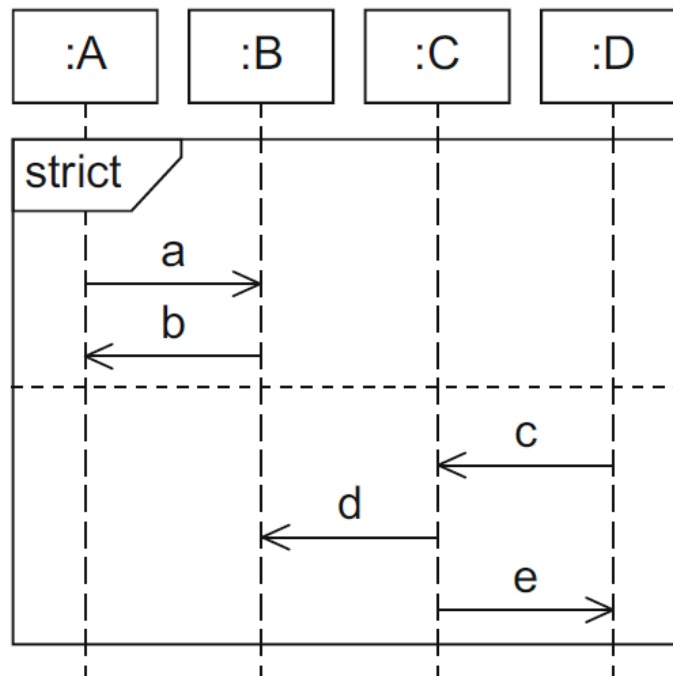
Example



- ▶ Note that seq is the default order and does not have to be modeled explicitly in this example.
- ▶ If there is no space, the student makes a reservation for the next day

Concurrency and order strict

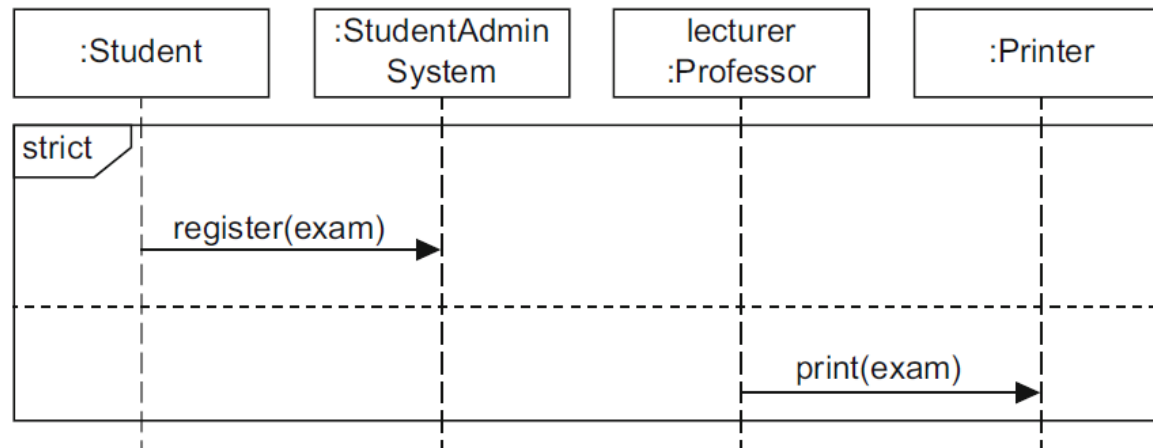
- ▶ The strict fragment describes a sequential interaction with a strict order.
- ▶ Even if there is no message exchange between the interaction partners, messages in an operand that is higher up on the vertical axis are always exchanged before the messages in an operand that is lower down on the vertical axis.



Traces:

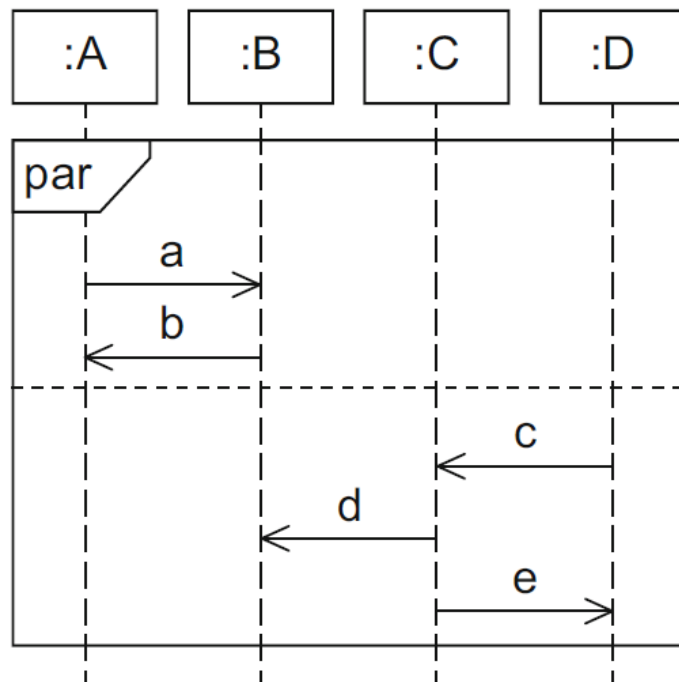
T01: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$

Example



Concurrency and order par

- ▶ The par fragment enables you to set aside any chronological order between messages in different operands.
- ▶ From a time perspective, the execution paths of the different operands can be interleaved as long as the restrictions of each individual operand are respected.
- ▶ Hence, the order of the different operands is irrelevant.
- ▶ However, this construct does not induce true parallelism, that is, it does not require simultaneous processing of the operands.
- ▶ The par operator actually expresses *concurrency*, that is, the order of the events that are located in different operands is irrelevant.



Traces:

T01: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$

T02: $a \rightarrow c \rightarrow b \rightarrow d \rightarrow e$

T03: $a \rightarrow c \rightarrow d \rightarrow b \rightarrow e$

T04: $a \rightarrow c \rightarrow d \rightarrow e \rightarrow b$

T05: $c \rightarrow a \rightarrow b \rightarrow d \rightarrow e$

T06: $c \rightarrow a \rightarrow d \rightarrow b \rightarrow e$

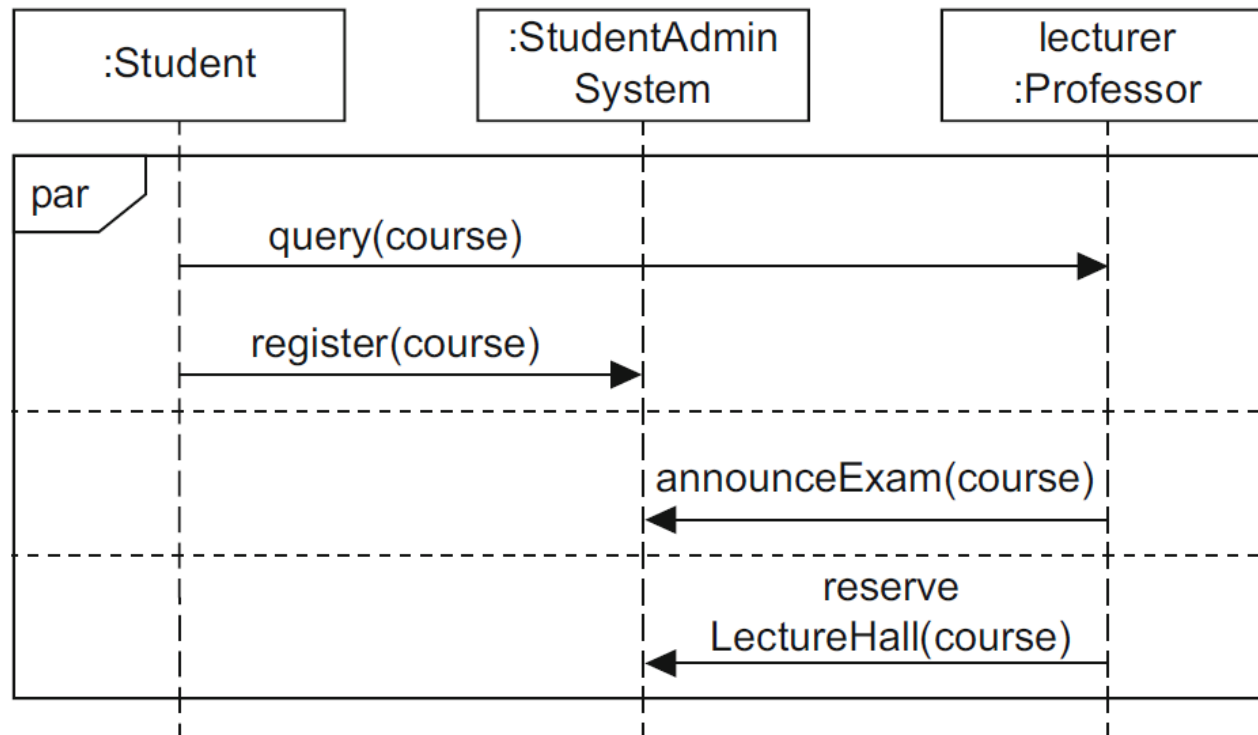
T07: $c \rightarrow a \rightarrow d \rightarrow e \rightarrow b$

T08: $c \rightarrow d \rightarrow a \rightarrow b \rightarrow e$

T09: $c \rightarrow d \rightarrow a \rightarrow e \rightarrow b$

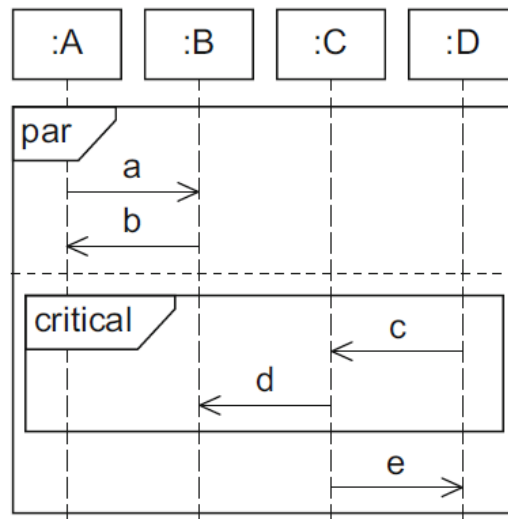
T10: $c \rightarrow d \rightarrow e \rightarrow a \rightarrow b$

Example



Concurrency and order critical

- ▶ To make sure that certain parts of an interaction are not interrupted by unexpected events, you can use the critical fragment. This marks an atomic area in the interaction.



Traces:

T01: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$

T02: $a \rightarrow c \rightarrow d \rightarrow b \rightarrow e$

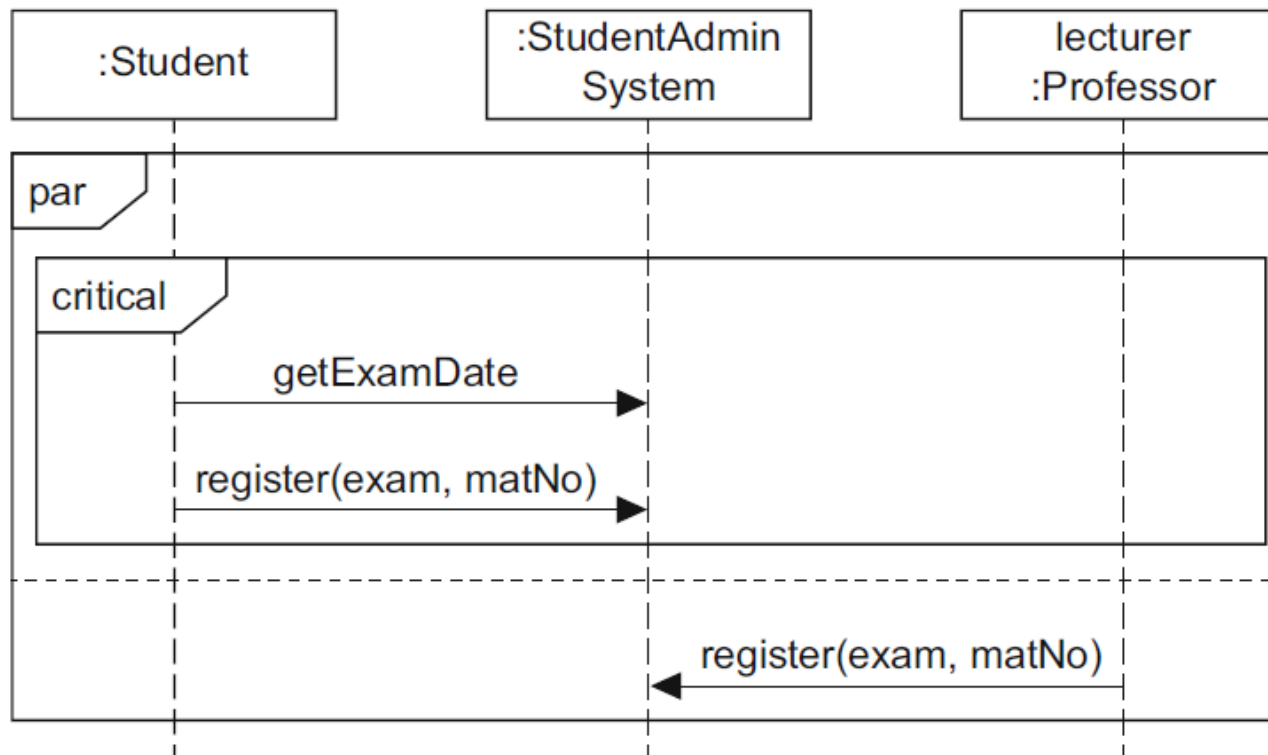
T03: $a \rightarrow c \rightarrow d \rightarrow e \rightarrow b$

T04: $c \rightarrow d \rightarrow a \rightarrow b \rightarrow e$

T05: $c \rightarrow d \rightarrow a \rightarrow e \rightarrow b$

T06: $c \rightarrow d \rightarrow e \rightarrow a \rightarrow b$

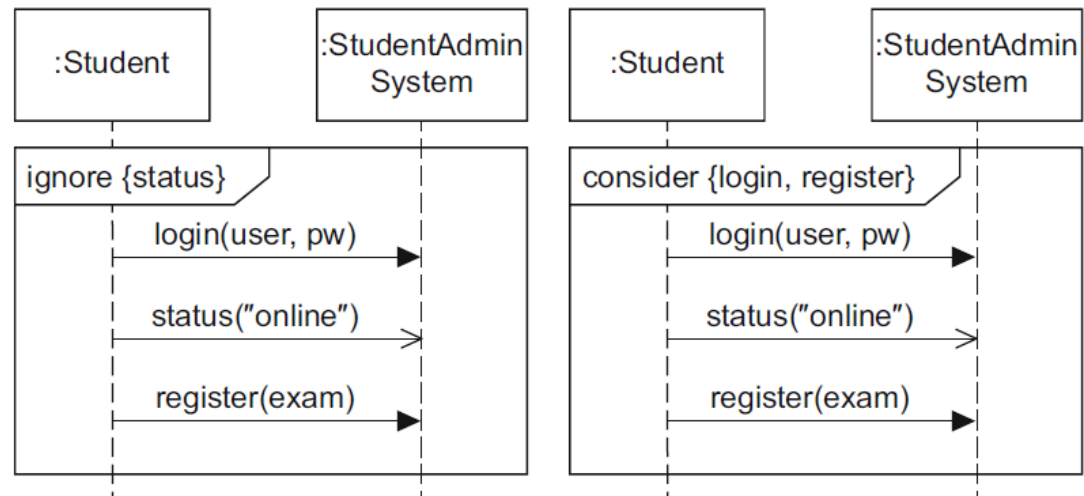
Example



Filters and Assertions

- ▶ The group “filters and assertions” define
 1. which messages may occur but are not relevant for the description of the system,
 2. which messages must occur,
 3. which messages must not occur. basing our explanation as closely as possible on the standard.
- ▶ Irrelevant messages are indicated by the ignore fragment, which expresses that these messages can occur at runtime but have no further significance for the functions depicted in the model.
- ▶ The consider fragment specifies those messages that are *consider* of particular importance for the interaction under consideration.
 - ▶ These messages are also shown in set notation after the keyword.
 - ▶ All messages that occur in the consider fragment but that are not specified in the set of relevant messages are automatically classified as irrelevant.

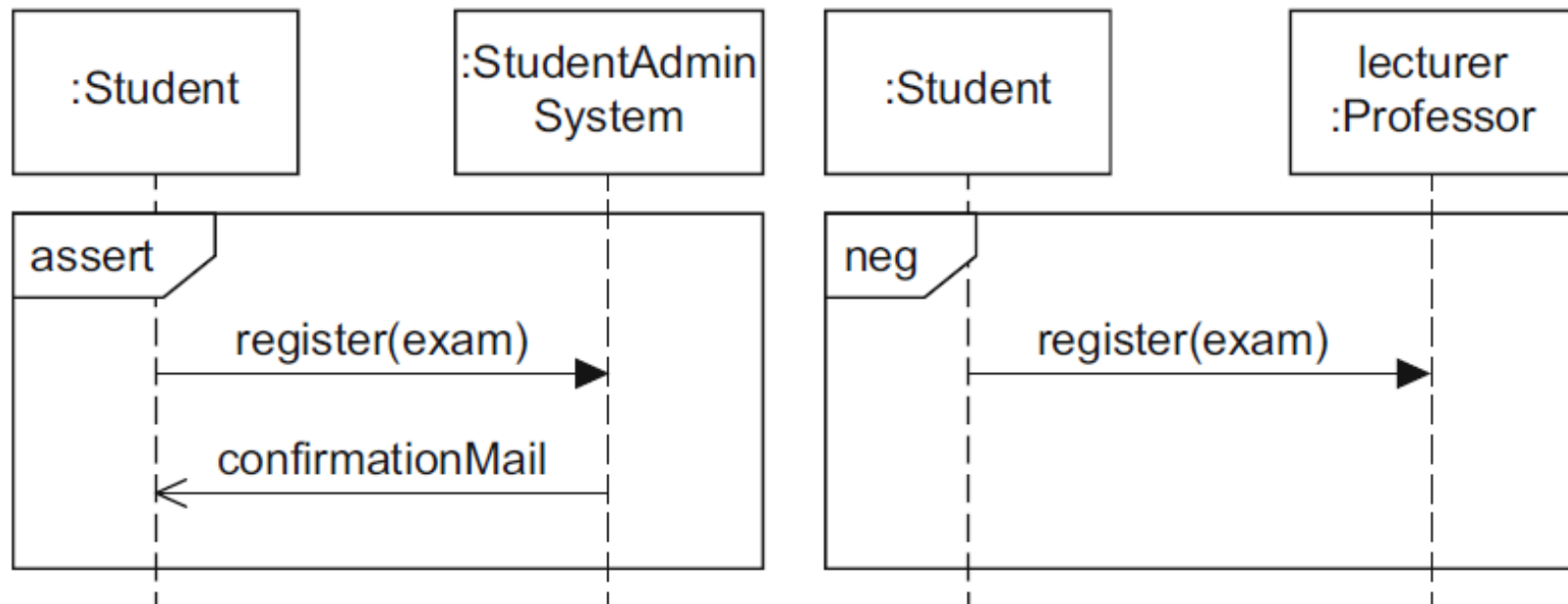
Example



Filters and Assertions

- ▶ The assert fragment identifies certain modeled traces as mandatory.
 - ▶ Deviations that occur in reality but that are not included in the diagram are not permitted.
 - ▶ This means that the implementation requires precise modeling and the model is a complete specification.
- ▶ With the neg fragment you model an invalid interaction
 - ▶ You describe situations that must not occur.
 - ▶ The neg fragment consists of exactly one operand.
 - ▶ You can use this fragment to explicitly highlight frequently occurring errors and to depict critical, incorrect sequences.
- ▶ However, there is no limit to the number of possible interaction sequences that should/must not occur, and so you must not assume that using the neg fragment will cover all undesirable situations.

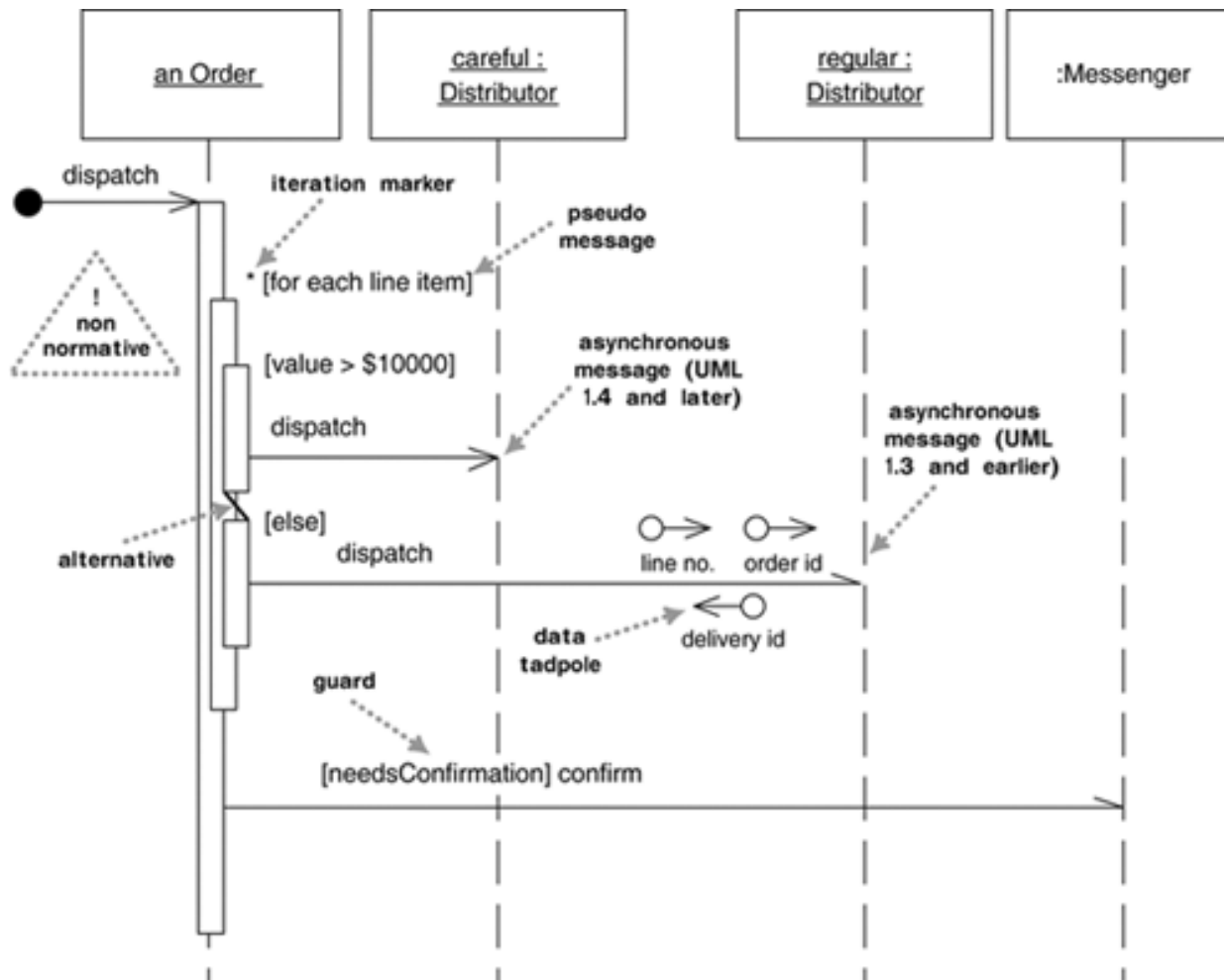
Example



Synchronous and Asynchronous Calls

- ▶ filled arrowheads show a synchronous message, while stick arrowheads show an asynchronous message.
 - ▶ If a caller sends a synchronous message, it must wait until the message is done, such as invoking a subroutine.
 - ▶ If a caller sends an asynchronous message, it can continue processing and doesn't have to wait for a response.
 - ▶ You see asynchronous calls in multithreaded applications and in message-oriented middleware.
 - ▶ Asynchrony gives better responsiveness and reduces the temporal coupling but is harder to debug.
- ▶ The arrowhead difference is very subtle.
 - ▶ It's also a backward-incompatible change introduced in UML 1.4, before then an asynchronous message was shown with the half-stick arrowhead.

Older Conventions



When to Use Sequence Diagrams

- ▶ You should use sequence diagrams when you want to look at the behavior of several objects within a single use case.
 - ▶ Sequence diagrams are good at showing collaborations among the objects; they are not so good at precise definition of the behavior.
 - ▶ If you want to look at the behavior of a single object across many use cases, use a state diagram.
 - ▶ If you want to look at behavior across many use cases or many threads, consider an activity diagram .
 - ▶ If you want to explore multiple alternative interactions quickly, you may be better off with CRC cards, as that avoids a lot of drawing and erasing.
- ▶ Other useful forms of interaction diagrams are communication diagrams, for showing connections; and timing diagrams, for showing timing constraints.

Sequence diagrams in Java

```

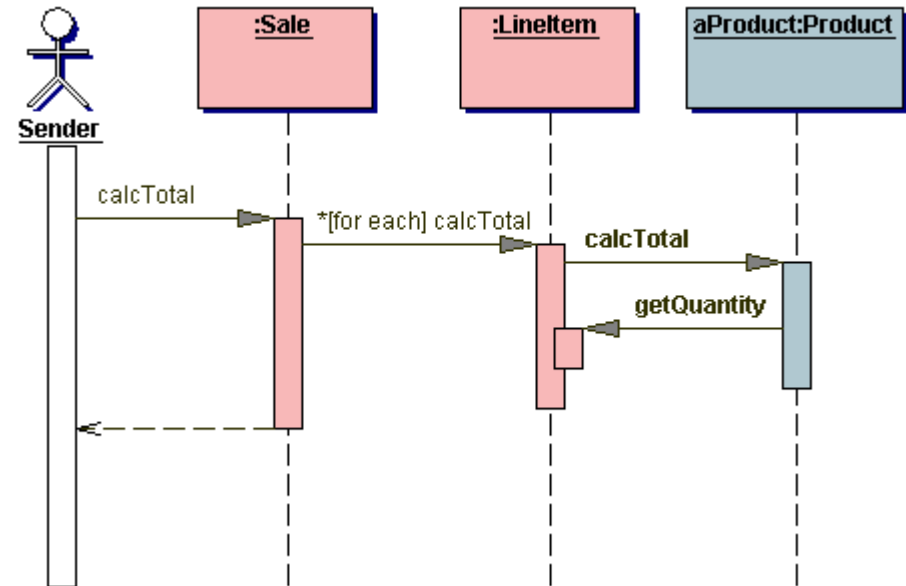
/**
 * From the Sale class:
 * calculates the total of the sale from the lineltem
subtotals
 * @return total of the sale
 */
public double calcTotal() {
    total = 0.0;
    Iterator i = lineltems.iterator();
    while (i.hasNext()) total +=
((LinItem)i.next()).calcTotal();
    return total;
}

```

```

/**
 * From the LinItem class:
 * calculates the cost of this amount of this kind of
item
 * @return the cost of this amount of the item
 */
public double calcTotal() {
    total = product.calcTotal(this);
    return total;
}

```

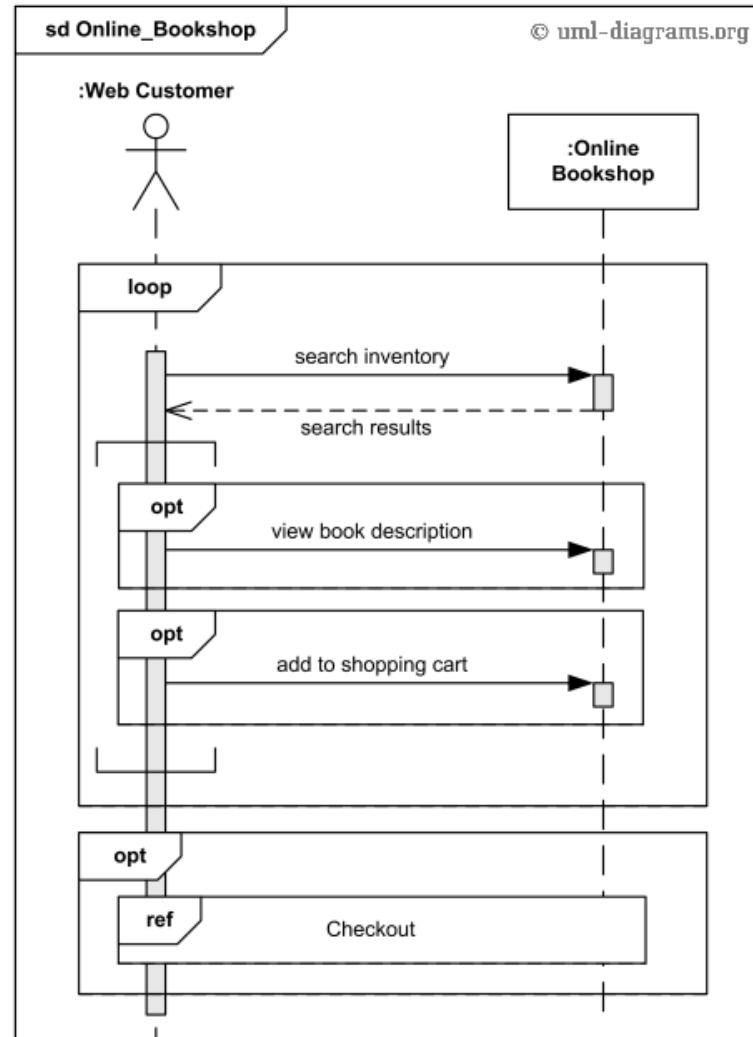


```

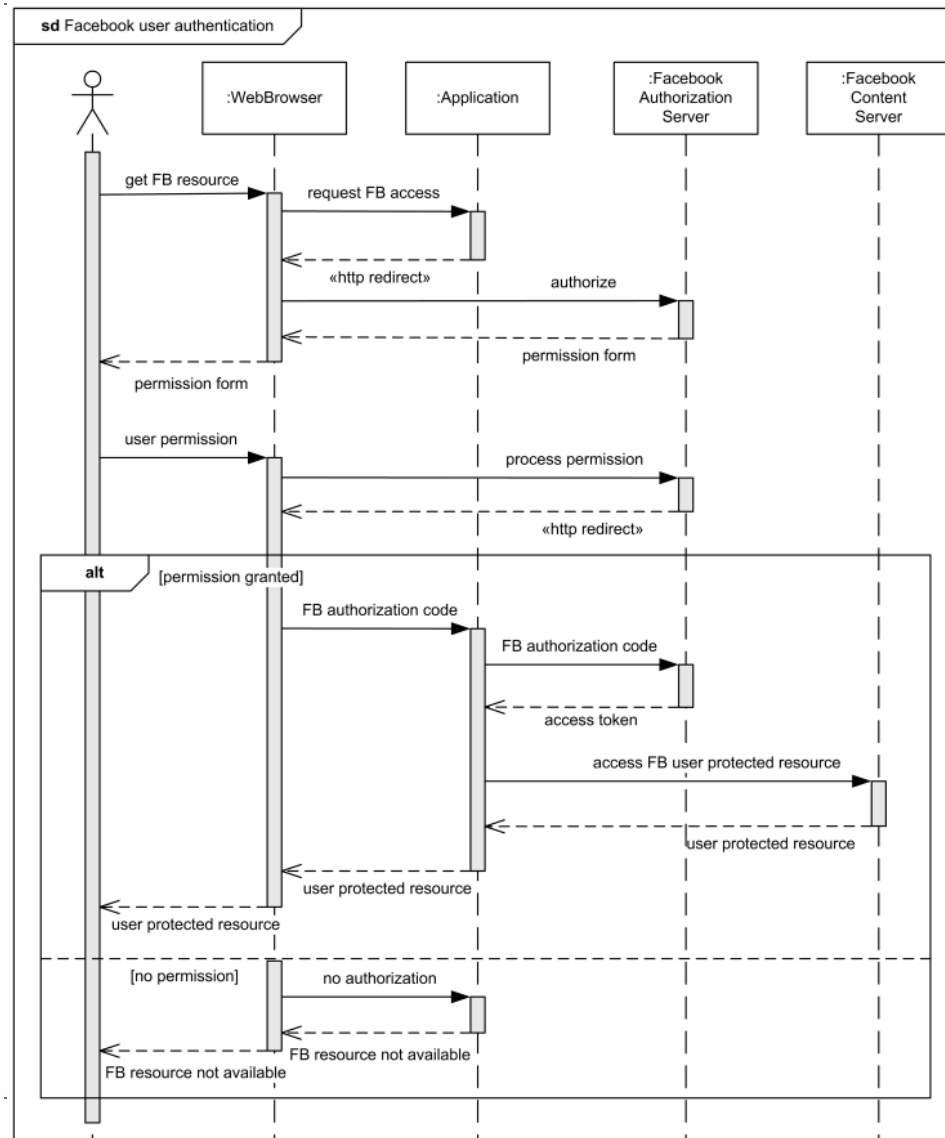
/**
 * From the Product class:
 * calculates the current cost of a quantity of the product
 * @return cost of the line item supplied
 */
public double calcTotal(LinItem li) {
    return amount * li.getQuantity();
}

```

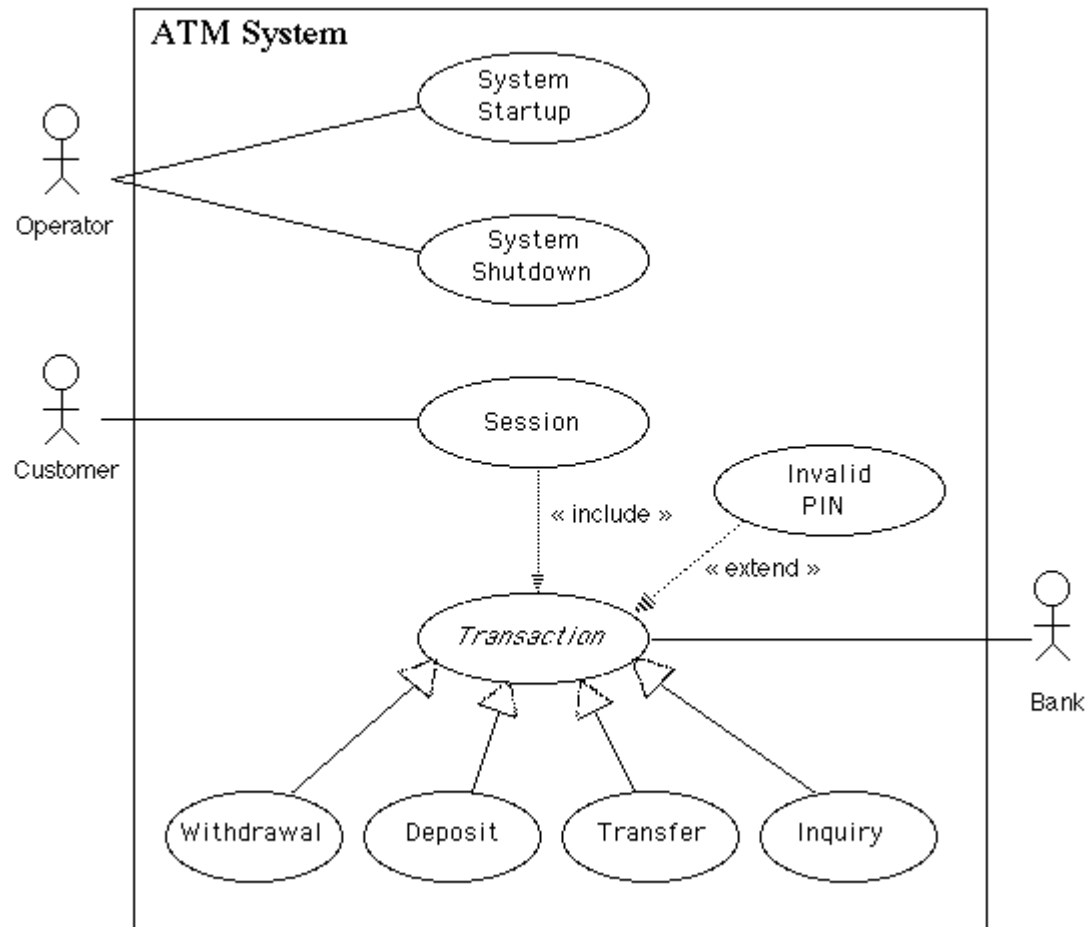
Example



Example



Example (ATM)

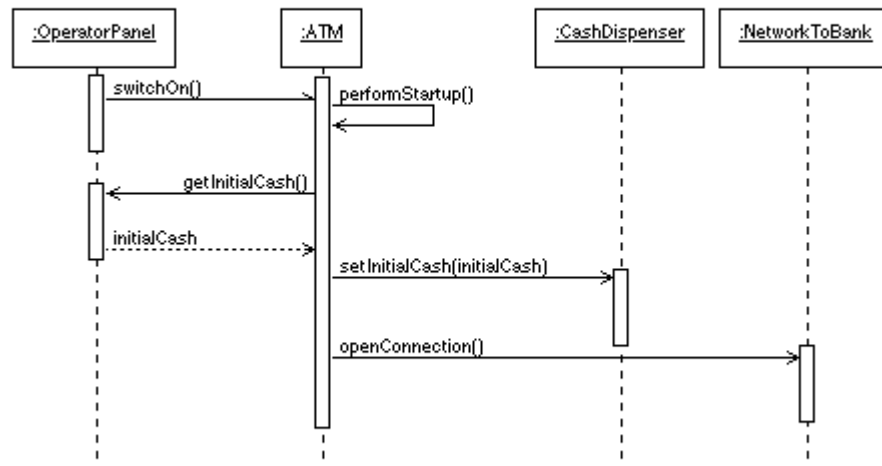


<http://www.math-cs.gordon.edu/courses/cs211/ATMExample/>

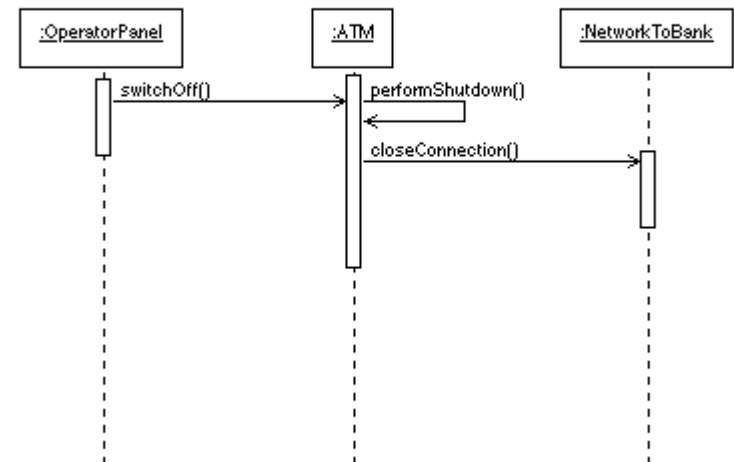


Example (ATM)

System Startup Sequence Diagram



System Shutdown Sequence Diagram

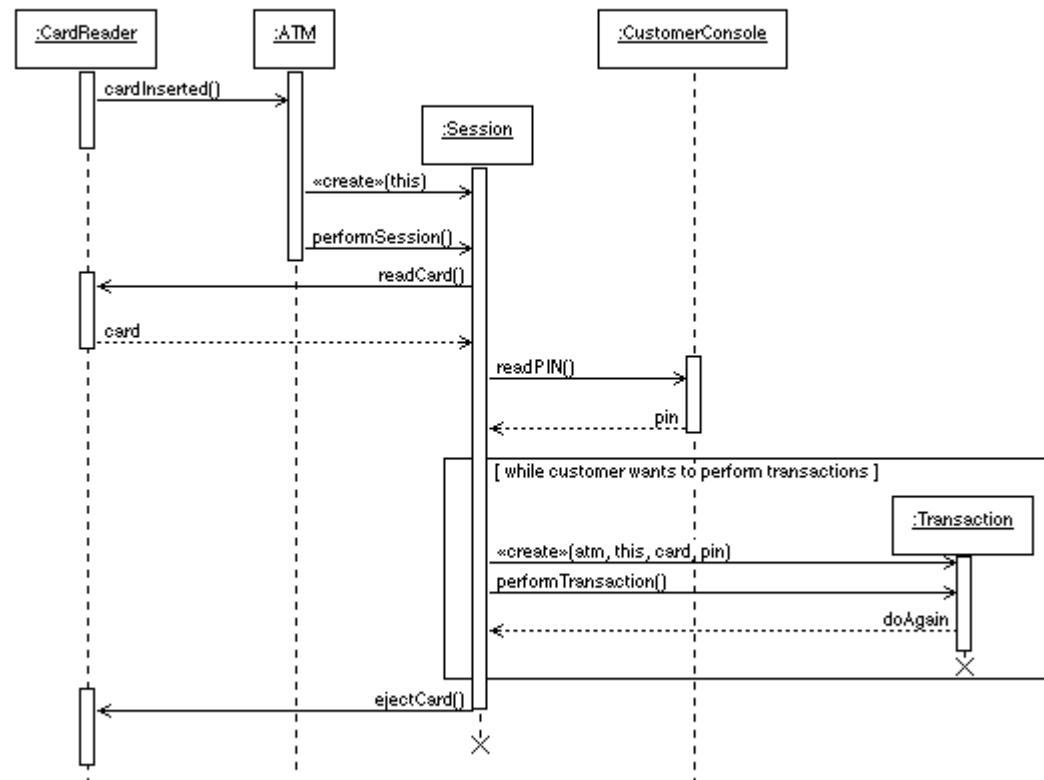


<http://www.math-cs.gordon.edu/courses/cs211/ATMExample/>



Example (ATM)

Session Sequence Diagram

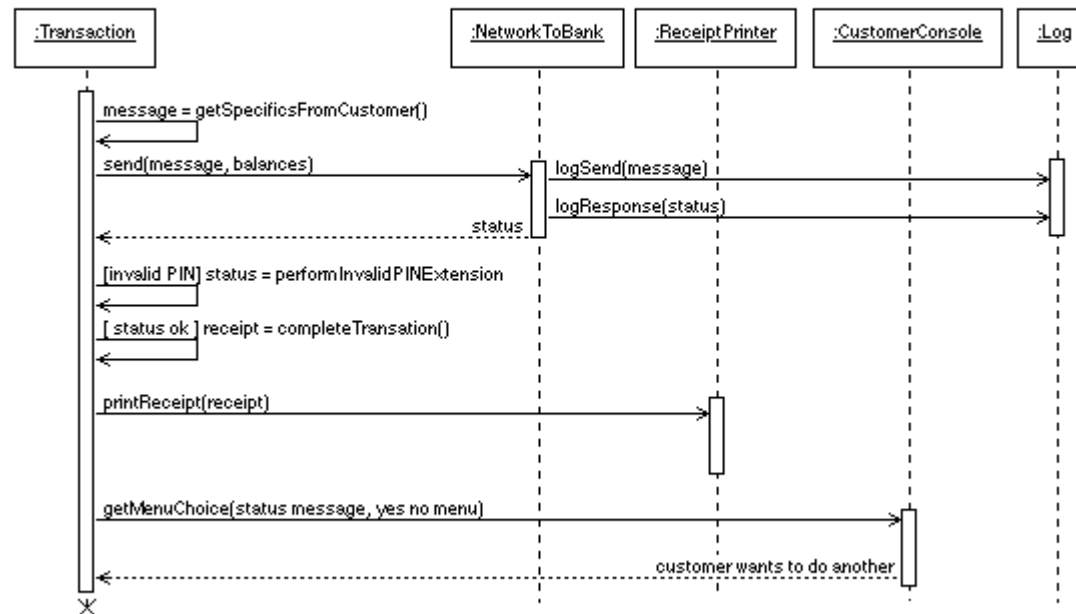


<http://www.math-cs.gordon.edu/courses/cs211/ATMExample/>



Example (ATM)

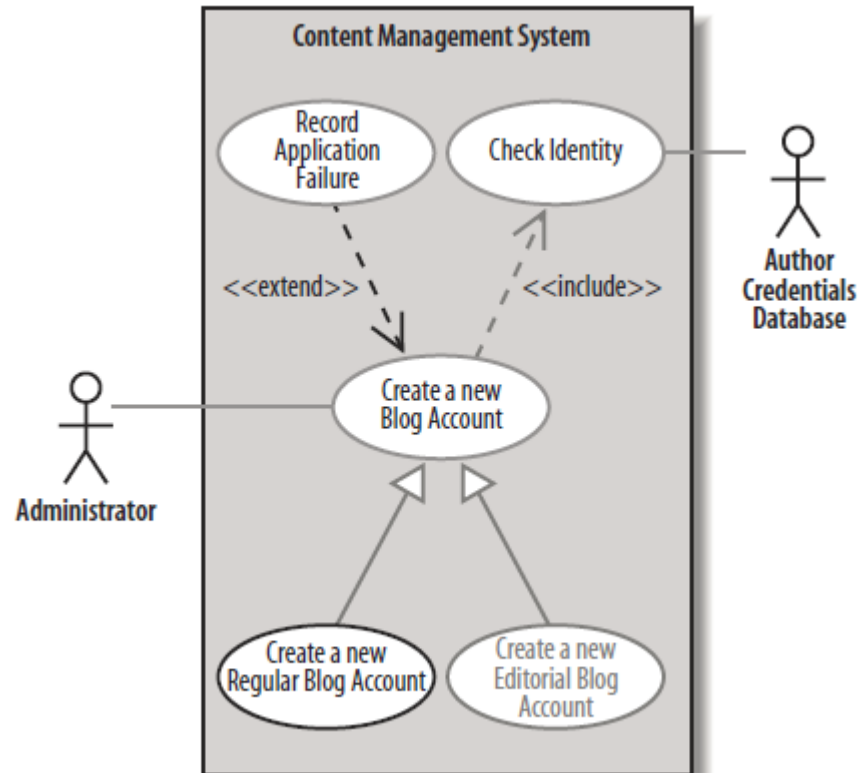
Transaction Sequence Diagram



<http://www.math-cs.gordon.edu/courses/cs211/ATMExample/>



Bringing a Use Case to Life with a Sequence Diagram



Main Flow

Step

Action

1

The Administrator asks the system to create a new blog account.

2

The Administrator selects the regular blog account type.

3

The Administrator enters the author's details.

4

The author's details are checked using the Author Credentials Database.

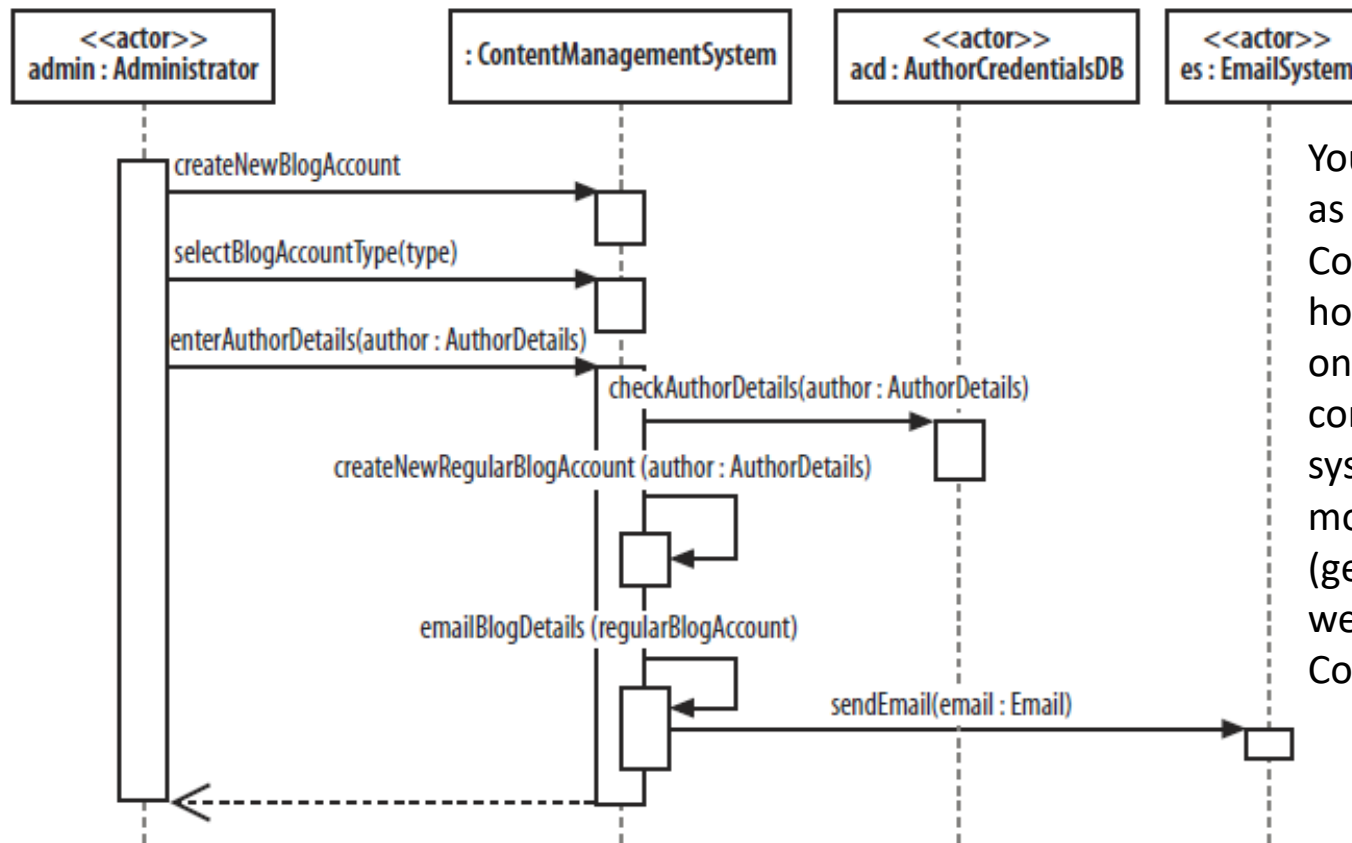
5

The new regular blog account is created.

6

A summary of the new blog account's details are emailed to the author.

Bringing a Use Case to Life with a Sequence Diagram (2)

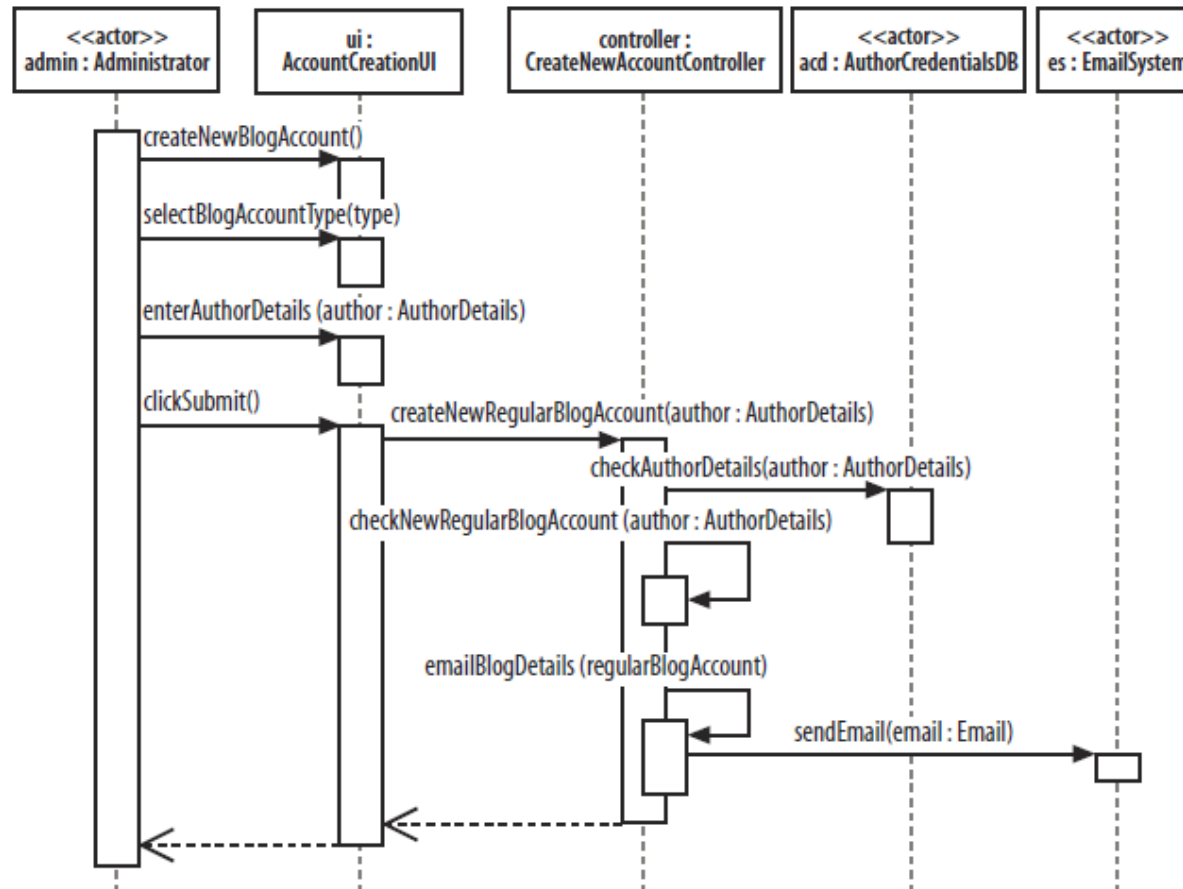


Your system is represented as a single participant: the ContentManagementSystem however, unless you intend on implementing your content management system as a single monolithic piece of code (generally not a good idea!), we have to break apart ContentManagementSystem

Main Flow	Step	Action
	1	The Administrator asks the system to create a new blog account.
	2	The Administrator selects the regular blog account type.
	3	The Administrator enters the author's details.
	4	The author's details are checked using the Author Credentials Database.
	5	The new regular blog account is created.
	6	A summary of the new blog account's details are emailed to the author.



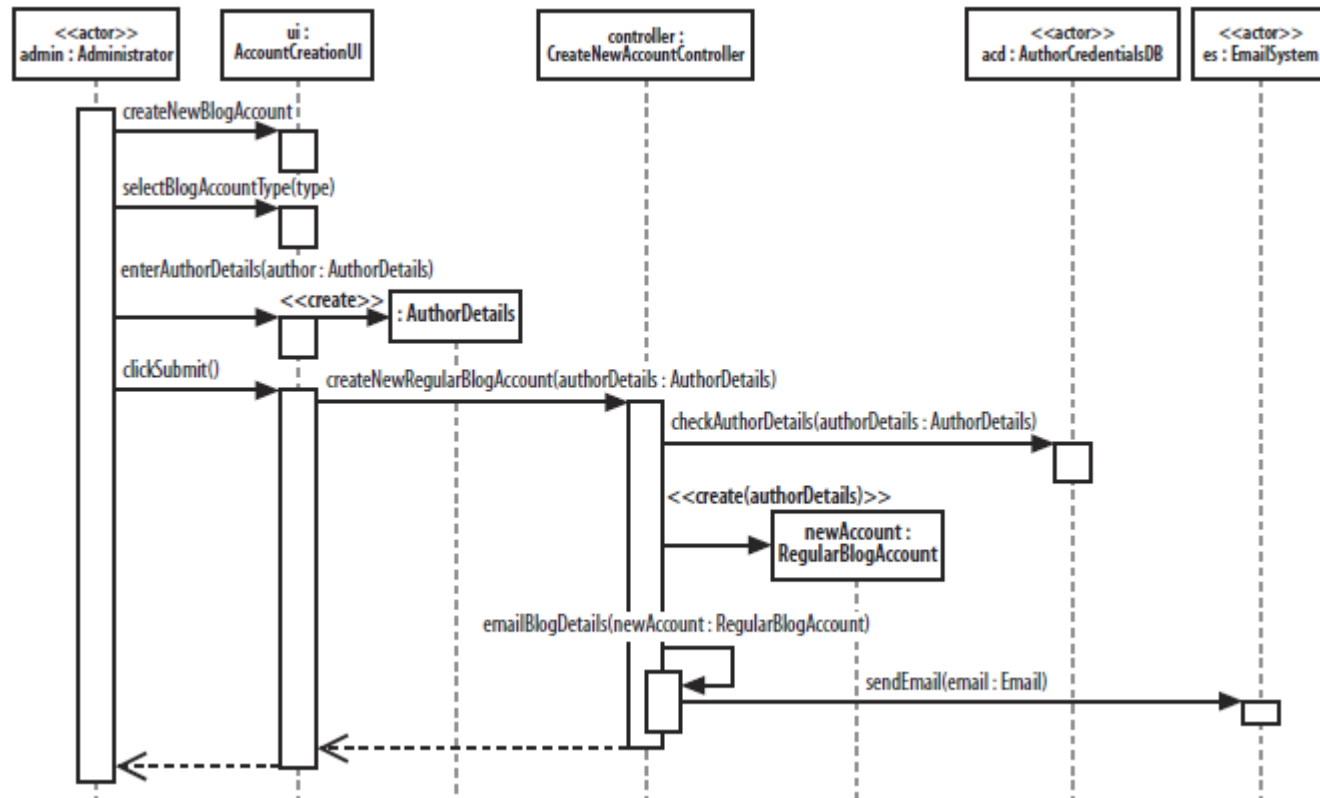
Breaking an Interaction into Separate Participants



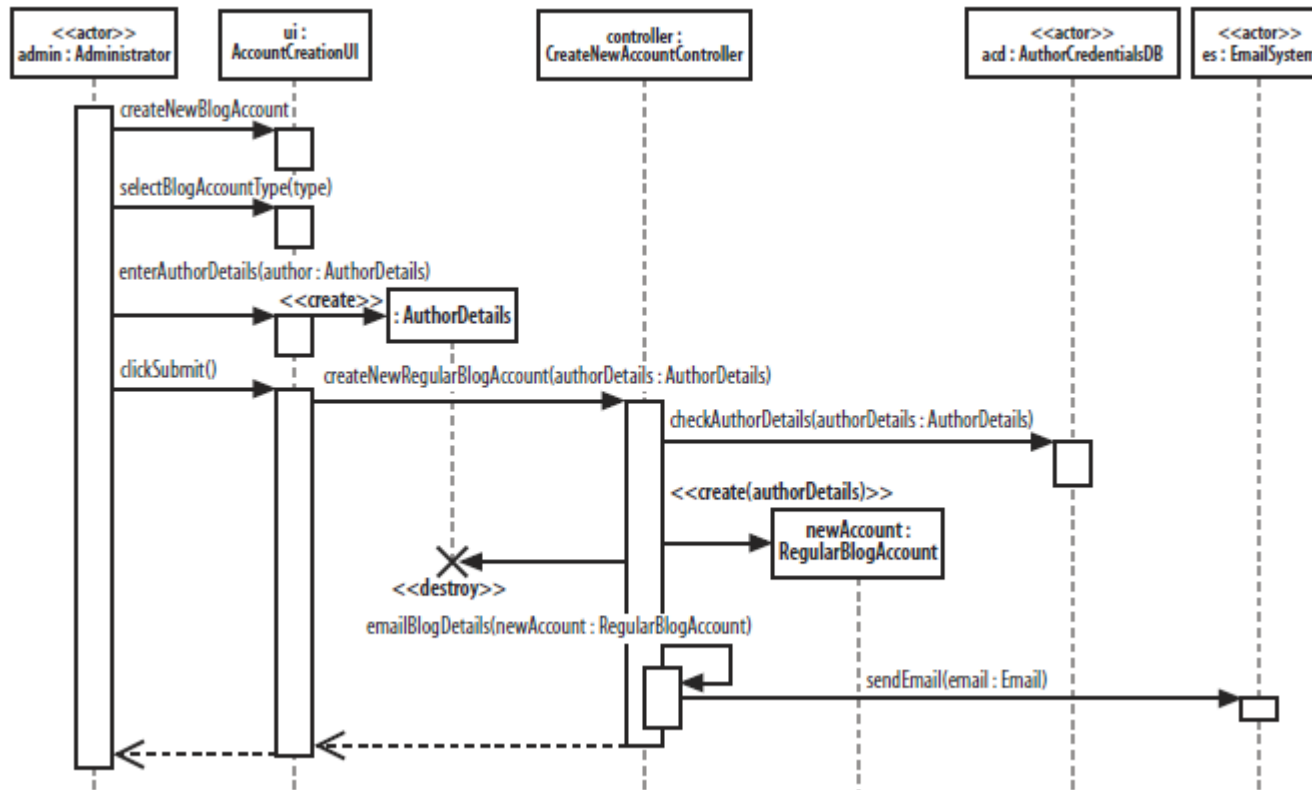
Something critical is missing from the sequence diagram.

The title of the use case in which the sequence diagram is operating is Create a new Regular Blog Account, but where is the actual *creation* of the blog account?

Applying Participant Creation



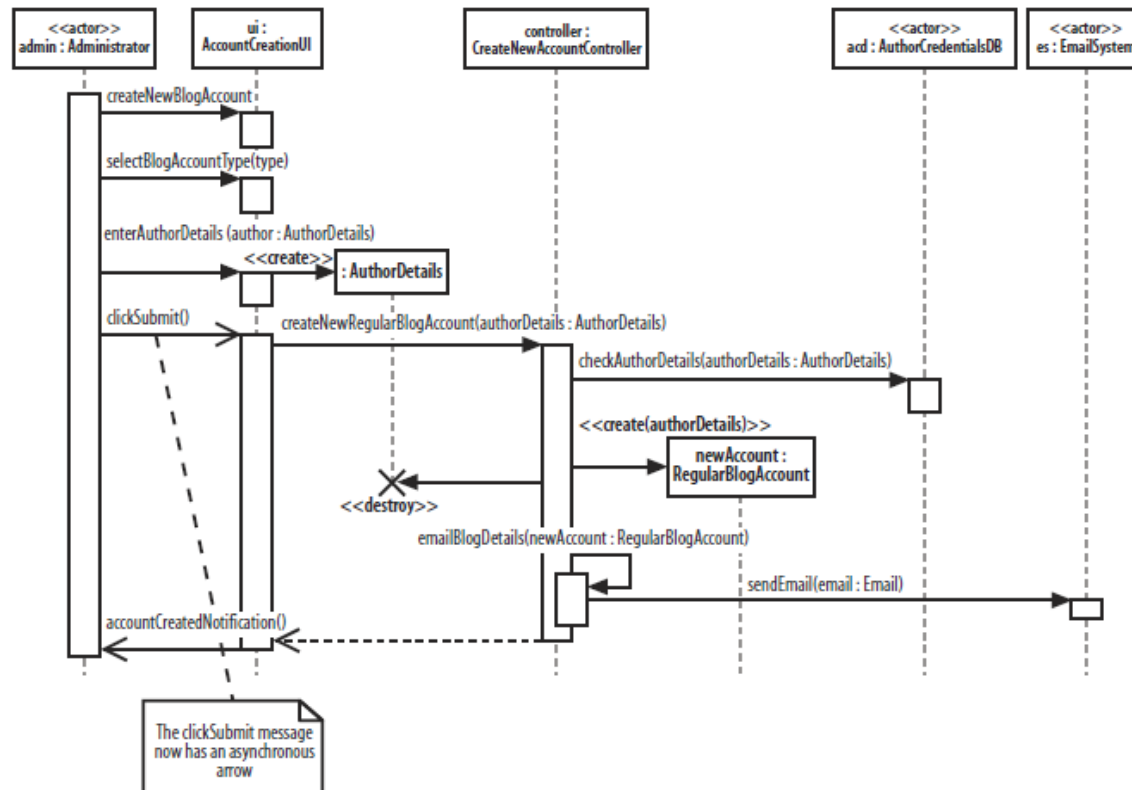
Applying Participant Deletion



When the Administrator clicks on the submit button the system freezes, until the new blog account has been created

the `authorDetails:AuthorDetails` participant is no longer required once the `newAccount:RegularBlogAccount` has been created.

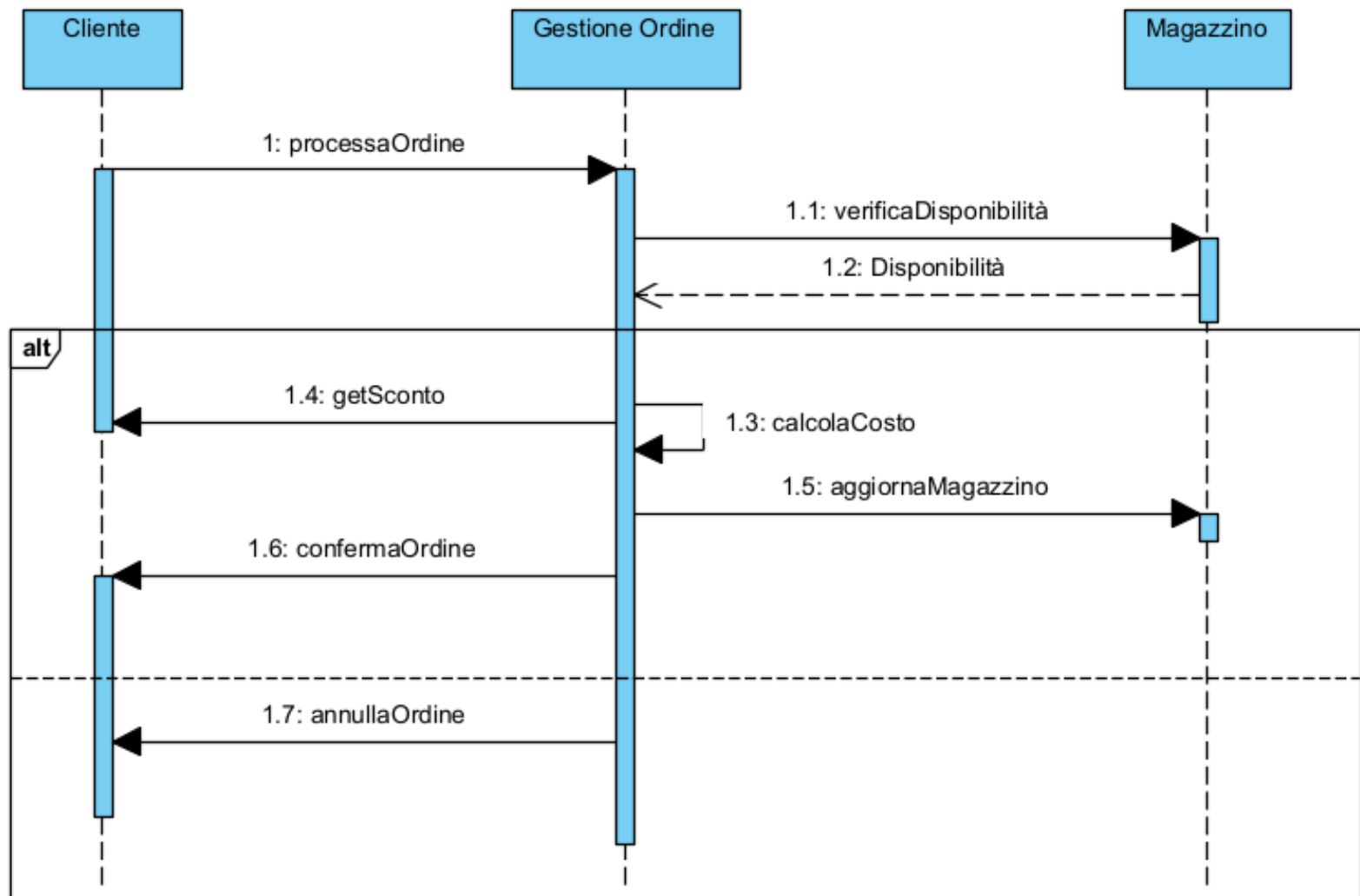
Applying Participant Deletion (2)



Converting the `clickSubmit()` from a synchronous to an asynchronous message means that the sequence diagram now shows that when the new regular blog account information is submitted, the user interface will not lock and wait for the new account to be created. Instead, the user interface allows the Administrator actor to continue working with the system.

Esercizio

Un cliente invia un ordine al sistema del negozio online. Il sistema verifica immediatamente la disponibilità del prodotto nel magazzino. Se il prodotto è disponibile, il sistema calcola il prezzo totale e applica eventuali sconti se il cliente dispone di un codice promozionale valido. Successivamente, invia una conferma d'ordine al cliente e aggiorna lo stato del magazzino. Se invece il prodotto non è disponibile, il sistema invia al cliente un messaggio di esaurito e annulla l'ordine.



Esercizio

- ▶ Si rappresenti il processo tipico per l'acquisto di due biglietti per assistere alla proiezione di un film. Si supponga che il cinema sia dotato di più sale, che il film scelto sia proiettato in una sola sala e possa avere orari diversi. Il diagramma deve considerare l'intero processo: dalla richiesta del cliente (found message) fino alla stampa dei biglietti.

Esercizio

- ▶ Si disegni il diagramma che descrive il calcolo dello stipendio di un dipendente di un'azienda. Lo scenario deve considerare un gestore che attraverso una query all'archivio aziendale crea un oggetto temporaneo dipendente. Questo viene usato per il calcolo dello stipendio e poi distrutto



References

- ▶ <http://www.math-cs.gordon.edu/courses/cs211/ATMExample/>
- ▶ Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language , Pearson Addison Wesley
- ▶ <http://www.edc4it.com/blog/java/implementing-a-bi-directional-association-in-java.html>
- ▶ Russ Miles, Kim Hamilton: Learning UML 2.0 A Pragmatic Introduction to UML, O'Reilly Media, 2006
- ▶ Martina Seidl, Marion Scholz, Christian Huemer, Gerti Kappel: UML @ Classroom - An Introduction to Object-Oriented Modeling, Springer 2015

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.